Stage 2 Implementation Report

PlanPilot — Adaptive Academic Scheduling Agent

1. Project Overview and Implementation Status

PlanPilot is a Java 17 Maven application for students who need help organizing academic work around deadlines and real availability. The submitted system provides both a JavaFX graphical user interface and a menu-driven command-line interface. Core planning behavior is deterministic: tasks are validated, prioritized, scheduled into valid study blocks, checked for conflicts, and updated through progress records. Gemini is integrated behind the LLMClient abstraction for natural-language task entry, effort-estimation assistance, workload-recovery suggestions, replanning recommendations, and schedule explanations.

The current implementation supports all eleven Stage 1 features: course management, task management, natural-language task entry, availability and commitment management, deterministic prioritization, study-schedule generation, conflict/workload recovery, progress tracking, assisted replanning, adaptive effort estimation, and schedule-decision explanation. User-facing IDs are generated internally rather than entered manually. Application state is stored locally so courses, tasks, availability, preferences, active schedule, and progress history can be restored after restart.

Limitations: PlanPilot is a single-user local desktop application and does not provide cloud accounts, login/signup, or cross-device synchronization. AI-assisted features require a locally configured Gemini API key and are subject to external model availability and rate/capacity limits. The deterministic planner continues to work without Gemini. AI suggestions are advisory and never silently modify deadlines, progress, estimates, or schedules.

2. Feature-to-Implementation Mapping

The following tables account for every Stage 1 feature and map each feature to the implemented classes, methods, design patterns, behavior type, and demonstration evidence.

F01 — Course Management

Feature ID and name	F01 — Course Management
Stage 1 feature/reference	F01 — Course Management

Status or change	Implemented
Implemented classes/files and responsibilities	Files: domain/Course.java, service/CourseService.java, controller/PlanPilotController.java, gui/PlanPilotGUI.java, cli/PlanPilotCLI.java. Classes: Course, CourseService, PlanPilotController, PlanPilotGUI, PlanPilotCLI. Responsibilities: Course stores course data; CourseService validates and manages course records; PlanPilotController exposes course operations; the GUI and CLI collect user input and display results.
Key methods	PlanPilotController.addCourse(), updateCourse(), deleteCourse(), getCourses(); CourseService.addCourse(), updateCourse(), deleteCourse()
Design patterns	MVC, Facade
Behavior type	Deterministic
Video link/timestamp	https://youtu.be/jssAO9gPK7A

Collaboration: The user invokes the course workflow from the GUI or CLI. The presentation layer sends the request to PlanPilotController, which delegates validation and storage to CourseService. The updated course list is returned to the interface. Duplicate codes and invalid updates are rejected deterministically.

F02 — Task Management

Feature ID and name	F02 — Task Management
Stage 1 feature/reference	F02 — Task Management
Status or change	Implemented
Implemented classes/files and responsibilities	Files: domain/Task.java, domain/AcademicTask.java, domain/GeneralTask.java, service/TaskService.java, controller/PlanPilotController.java, gui/PlanPilotGUI.java, cli/PlanPilotCLI.java. Classes: Task, AcademicTask, GeneralTask, TaskService, PlanPilotController, PlanPilotGUI, PlanPilotCLI. Responsibilities: Task defines common task state; AcademicTask

	and GeneralTask model the two task types; TaskService validates and stores tasks; PlanPilotController coordinates CRUD operations; GUI and CLI collect task input and display task state.
Key methods	TaskService.createTask(), updateTask(), deleteTask(), getOpenTasks(); PlanPilotController.addTask(), updateTask(), deleteTask(), getTasks()
Design patterns	MVC, Facade
Behavior type	Deterministic
Video link/timestamp	https://youtu.be/C_jMAy0V6tg

Collaboration: The user enters task data in the GUI or CLI. PlanPilotController forwards the request to TaskService, which validates the task and stores either an AcademicTask or GeneralTask. Academic-only fields are shown only when the task kind is Academic. Task changes mark an existing schedule outdated rather than silently regenerating it.

F03 — Natural-Language Task Entry

Feature ID and name	F03 — Natural-Language Task Entry
Stage 1 feature/reference	F03 — Natural-Language Task Entry
Status or change	Modified — implemented with Gemini instead of the Stage 1 OpenAI adapter
Implemented classes/files and responsibilities	Files: ai/PlanningAgent.java, ai/GeminiLLMClient.java, ai/LLMClient.java, ai/TaskParser.java, ai/ParsedTaskData.java, controller/PlanPilotController.java, gui/PlanPilotGUI.java, cli/PlanPilotCLI.java. Classes/interfaces: PlanningAgent, GeminiLLMClient, LLMClient, TaskParser, ParsedTaskData, PlanPilotController, PlanPilotGUI, PlanPilotCLI. Responsibilities: PlanningAgent builds the AI request; GeminiLLMClient adapts Gemini to LLMClient; TaskParser parses and validates structured output; ParsedTaskData carries the proposal; PlanPilotController exposes the workflow; GUI/CLI show the proposal for user review before saving.
Key methods	PlanPilotController.parseNaturalLanguageTask(); PlanningAgent.parseTask(); LLMClient.generateText(); TaskParser.parse()

Design patterns	Adapter, MVC, Facade
Behavior type	AI-assisted with deterministic validation and user confirmation
Video link/timestamp	https://youtu.be/NFJIo7skSKY

Collaboration: The user provides free-form task text. PlanPilotController delegates interpretation to PlanningAgent, which calls Gemini through the LLMClient abstraction. TaskParser converts the response into validated ParsedTaskData. The interface fills a proposal form, and the user must explicitly review and save the task.

F04 — Availability and Commitment Management

Feature ID and name	F04 — Availability and Commitment Management
Stage 1 feature/reference	F04 — Availability and Commitment Management
Status or change	Implemented
Implemented classes/files and responsibilities	Files: domain/AvailabilityBlock.java, service/AvailabilityService.java, controller/PlanPilotController.java, gui/PlanPilotGUI.java, cli/PlanPilotCLI.java. Classes: AvailabilityBlock, AvailabilityService, PlanPilotController, PlanPilotGUI, PlanPilotCLI. Responsibilities: AvailabilityBlock represents available study time or fixed commitments; AvailabilityService validates and manages time entries; PlanPilotController exposes CRUD operations; GUI/CLI collect and display dates, times, and availability types.
Key methods	AvailabilityService.addAvailability(), updateAvailability(), deleteAvailability(), getAvailability(); PlanPilotController.addAvailability(), updateAvailability(), deleteAvailability()
Design patterns	MVC, Facade
Behavior type	Deterministic
Video link/timestamp	https://youtu.be/TjN4LXFN8jw

Collaboration: The user adds available study windows or fixed commitments. PlanPilotController delegates to AvailabilityService, which validates each interval and rejects contradictory overlap

between available and unavailable time. Schedule generation later consumes these stored blocks.

F05 — Task Prioritization

Feature ID and name	F05 — Task Prioritization
Stage 1 feature/reference	F05 — Task Prioritization
Status or change	Implemented
Implemented classes/files and responsibilities	Files: strategy/PriorityStrategy.java, strategy/BalancedPriorityStrategy.java, service/SchedulePlanner.java. Classes/interfaces: PriorityStrategy, BalancedPriorityStrategy, SchedulePlanner. Responsibilities: PriorityStrategy defines the replaceable priority algorithm; BalancedPriorityStrategy calculates deterministic scores and factor contributions; SchedulePlanner uses the strategy to order open tasks before allocating study time.
Key methods	PriorityStrategy.calculatePriority(), evaluate(); BalancedPriorityStrategy.calculatePriority(), evaluate(); SchedulePlanner.createSchedule()
Design patterns	Strategy
Behavior type	Deterministic
Video link/timestamp	https://youtu.be/mc1_EMCXaBg

Collaboration: Before allocating study time, SchedulePlanner asks the configured PriorityStrategy to evaluate each open task. BalancedPriorityStrategy combines deadline urgency, remaining effort, difficulty, user importance, unfinished proportion, and academic assessment weight. The resulting deterministic ordering is used by the planner.

F06 — Study Schedule Generation

Feature ID and name	F06 — Study Schedule Generation
Stage 1 feature/reference	F06 — Study Schedule Generation

Status or change	Implemented
Implemented classes/files and responsibilities	Files: service/ScheduleService.java, service/SchedulePlanner.java, service/ConflictDetector.java, domain/Schedule.java, domain/StudyBlock.java, domain/UserPreferences.java, controller/PlanPilotController.java. Classes: ScheduleService, SchedulePlanner, ConflictDetector, Schedule, StudyBlock, UserPreferences, PlanPilotController. Responsibilities: ScheduleService gathers planning state and coordinates generation; SchedulePlanner allocates StudyBlocks; ConflictDetector checks feasibility; Schedule stores the generated plan; StudyBlock represents each session; UserPreferences supplies session/break/preferred-period settings; PlanPilotController exposes generation to the interfaces.
Key methods	PlanPilotController.generateSchedule(); ScheduleService.generateSchedule(); SchedulePlanner.createSchedule(); ConflictDetector.isScheduleFeasible()
Design patterns	Facade, Strategy
Behavior type	Deterministic
Video link/timestamp	https://youtu.be/EfaGdMDhVTI

Collaboration: The user selects a planning window and requests generation. PlanPilotController calls ScheduleService, which retrieves open tasks and availability and delegates allocation to SchedulePlanner. The planner orders tasks through PriorityStrategy, creates StudyBlocks inside valid time, applies preferences, and checks feasibility before returning the schedule.

F07 — Schedule Conflict and Workload Recovery

Feature ID and name	F07 — Schedule Conflict and Workload Recovery
Stage 1 feature/reference	F07 — Schedule Conflict and Workload Recovery
Status or change	Modified — deterministic deficit detection plus explicit recovery actions and optional Gemini advice
Implemented classes/files and responsibilities	Files: service/SchedulePlanner.java, service/ScheduleService.java, service/ScheduleInfeasibleException.java, service/RecoveryRequest.java, ai/PlanningAgent.java,

	ai/RecoveryPlan.java, controller/PlanPilotController.java. Classes: SchedulePlanner, ScheduleService, ScheduleInfeasibleException, RecoveryRequest, PlanningAgent, RecoveryPlan, PlanPilotController. Responsibilities: SchedulePlanner/ScheduleService detect infeasible workload and preserve a partial plan; ScheduleInfeasibleException reports the workload deficit; PlanningAgent generates optional recovery advice; RecoveryPlan carries AI suggestions; RecoveryRequest represents the user's chosen recovery action; PlanPilotController separates advice from explicit application.
Key methods	ScheduleService.generateSchedule(), requireRecoverySchedule(), replan(); PlanPilotController.requestRecoverySuggestions(), applyRecovery(); PlanningAgent.suggestRecovery()
Design patterns	Facade, Adapter, Strategy
Behavior type	Hybrid
Video link/timestamp	https://youtu.be/F9MEcAeznek

Collaboration: If deterministic generation cannot fit all required work, the scheduler returns a partial plan and workload deficit rather than inventing overlapping sessions. The user may request Gemini recovery advice. The AI only proposes options; a separate user-confirmed RecoveryRequest is required before availability, window length, or deferral is changed and the deterministic scheduler runs again.

F08 — Progress Tracking and Inferred Completion

Feature ID and name	F08 — Progress Tracking and Inferred Completion
Stage 1 feature/reference	F08 — Progress Tracking and Inferred Completion
Status or change	Implemented
Implemented classes/files and responsibilities	Files: service/ProgressService.java, domain/ProgressRecord.java, domain/ProgressStatus.java, domain/StudyBlock.java, controller/PlanPilotController.java, gui/PlanPilotGUI.java, cli/PlanPilotCLI.java. Classes/enums: ProgressService, ProgressRecord, ProgressStatus, StudyBlock, PlanPilotController, PlanPilotGUI,

	PlanPilotCLI. Responsibilities: ProgressService tracks sessions, inferred outcomes, corrections, and history; ProgressRecord stores progress data; ProgressStatus represents completion state; StudyBlock identifies the session being reported; PlanPilotController exposes progress operations; GUI/CLI let the user confirm, partially complete, skip, or correct sessions.
Key methods	PlanPilotController.updateProgress(), inferProgress(); ProgressService.recordProgress(), inferProgress(), getHistoryForTask(), trackSessions()
Design patterns	Observer, Facade
Behavior type	Deterministic
Video link/timestamp	https://youtu.be/xYJjJAr2yK0

Collaboration: The interface presents scheduled StudyBlocks. User-confirmed outcomes are sent through PlanPilotController to ProgressService, which records or corrects the session contribution and updates remaining task effort. Elapsed sessions may be tentatively inferred, but the user can correct them. Relevant changes notify observers and can mark the schedule outdated.

F09 — Assisted Dynamic Replanning

Feature ID and name	F09 — Assisted Dynamic Replanning
Stage 1 feature/reference	F09 — Assisted Dynamic Replanning
Status or change	Implemented with advisory Gemini recommendation
Implemented classes/files and responsibilities	Files: service/ScheduleService.java, service/ReplanOption.java, service/SchedulePlanner.java, ai/PlanningAgent.java, ai/ReplanRecommendation.java, controller/PlanPilotController.java. Classes/enums: ScheduleService, ReplanOption, SchedulePlanner, PlanningAgent, ReplanRecommendation, PlanPilotController. Responsibilities: ScheduleService owns current schedule state and executes deterministic replanning; ReplanOption defines allowed choices; SchedulePlanner generates the revised plan; PlanningAgent may recommend an option; ReplanRecommendation carries advisory AI output; PlanPilotController keeps recommendation separate from explicit application.

Key methods	PlanPilotController.requestReplan(), requestReplanRecommendation(); ScheduleService.replan(); PlanningAgent.suggestReplanningOption()
Design patterns	Facade, Strategy, Adapter, Observer
Behavior type	Hybrid
Video link/timestamp	https://youtu.be/huAK5cdWDDM

Collaboration: Task, progress, preference, or availability changes can mark a schedule outdated. The user may ask Gemini for a recommendation, but no schedule changes occur at that point. Only an explicit user-selected ReplanOption reaches ScheduleService, which invokes the deterministic planner using the latest state.

F10 — Adaptive Effort Estimation

Feature ID and name	F10 — Adaptive Effort Estimation
Stage 1 feature/reference	F10 — Adaptive Effort Estimation / Effort Estimation Feedback
Status or change	Modified — AI estimation was integrated directly into the Tasks workflow
Implemented classes/files and responsibilities	Files: service/EffortEstimator.java, service/ProgressService.java, ai/PlanningAgent.java, ai/LLMClient.java, ai/GeminiLLMClient.java, controller/PlanPilotController.java, gui/PlanPilotGUI.java. Classes/interfaces: EffortEstimator, ProgressService, PlanningAgent, LLMClient, GeminiLLMClient, PlanPilotController, PlanPilotGUI. Responsibilities: ProgressService provides historical progress; EffortEstimator identifies/calculates relevant historical context; PlanningAgent requests an advisory estimate through LLMClient/GeminiLLMClient; PlanPilotController exposes the operation; PlanPilotGUI lets the user review, keep, or apply the suggestion without automatically saving the task.
Key methods	PlanPilotController.requestEffortSuggestion(); EffortEstimator.estimateEffort(); PlanningAgent.suggestEffort(); ProgressService.getHistory()
Design patterns	Facade, Adapter, MVC

Behavior type	Hybrid
Video link/timestamp	https://youtu.be/D1yfUTm5tgk

Collaboration: While creating or editing a task, the user requests an estimate. The controller sends task details and relevant history to EffortEstimator and PlanningAgent. Gemini returns a suggestion, but the current estimate is unchanged until the user chooses to apply it. Applying only edits the form; Add Task or Update remains a separate explicit save action.

F11 — Schedule Decision Explanation

Feature ID and name	F11 — Schedule Decision Explanation
Stage 1 feature/reference	F11 — Schedule Decision Explanation
Status or change	Modified — implemented with Gemini and richer persisted scheduling evidence
Implemented classes/files and responsibilities	Files: service/SchedulePlanner.java, service/ScheduleService.java, strategy/BalancedPriorityStrategy.java, ai/PlanningAgent.java, ai/LLMClient.java, ai/GeminiLLMClient.java, controller/PlanPilotController.java. Classes/interfaces: SchedulePlanner, ScheduleService, BalancedPriorityStrategy, PlanningAgent, LLMClient, GeminiLLMClient, PlanPilotController. Responsibilities: SchedulePlanner records generation evidence; BalancedPriorityStrategy supplies priority scores/factor contributions; ScheduleService retains the current generated schedule; PlanningAgent builds a read-only grounded explanation context; GeminiLLMClient produces the natural-language explanation through LLMClient; PlanPilotController exposes the request without allowing AI to modify schedule state.
Key methods	PlanPilotController.explainSchedule(); PlanningAgent.explainSchedule(); BalancedPriorityStrategy.evaluate(); LLMClient.generateText()
Design patterns	Adapter, Facade, MVC, Strategy
Behavior type	AI explanation grounded in deterministic state
Video link/timestamp	https://youtu.be/O5jMsr6YMPo

Collaboration: The user asks a question about the current plan. PlanPilotController obtains the existing schedule context and passes it to PlanningAgent. The context includes deterministic task, priority, availability, and placement facts. Gemini explains those facts in natural language; it cannot regenerate the schedule, recalculate priorities, or modify application state.

3. Implemented Design Patterns

3.1 Model-View-Controller (MVC)

Design problem: PlanPilot must support both JavaFX and CLI interfaces without placing scheduling, validation, persistence, or AI logic inside presentation code.
Participants and roles: PlanPilotGUI and PlanPilotCLI are views/boundary interfaces; PlanPilotController coordinates user requests; domain classes and services form the model/application layer.
Relevant files and methods: PlanPilotGUI, PlanPilotCLI, PlanPilotController, and service classes such as TaskService and ScheduleService.
Why useful: MVC keeps interface code separate from application behavior, allows GUI and CLI to reuse the same controller/services, and makes the scheduling system easier to test.
Stage 1 alignment: Matches Stage 1. The Stage 2 GUI was simplified and expanded, but the MVC responsibility split was preserved.

3.2 Facade

Design problem: The GUI and CLI would otherwise need direct knowledge of many independent services.
Participants and roles: PlanPilotController is the Facade; CourseService, TaskService, AvailabilityService, ScheduleService, ProgressService, EffortEstimator, and PlanningAgent are subsystem components.
Relevant methods: addCourse(), addTask(), generateSchedule(), updateProgress(), requestReplan(), requestEffortSuggestion(), parseNaturalLanguageTask(), explainSchedule(), and AI recovery/replan methods.
Why useful: Both interfaces communicate through one stable application-facing API while internal services remain cohesive.
Stage 1 alignment: Matches Stage 1.

3.3 Strategy

Design problem: Task-priority calculation should be replaceable without rewriting SchedulePlanner.

Participants and roles: PriorityStrategy defines the algorithm contract; BalancedPriorityStrategy implements the current formula; SchedulePlanner depends on PriorityStrategy rather than a concrete scoring algorithm.
Relevant methods: PriorityStrategy.calculatePriority(), PriorityStrategy.evaluate(), BalancedPriorityStrategy.evaluate(), SchedulePlanner.createSchedule().
Why useful: Priority logic can evolve independently from allocation logic and is testable as a separate behavior.
Stage 1 alignment: Matches Stage 1. Stage 2 also records the individual score contributions so schedule explanations can be grounded in the actual generated plan.

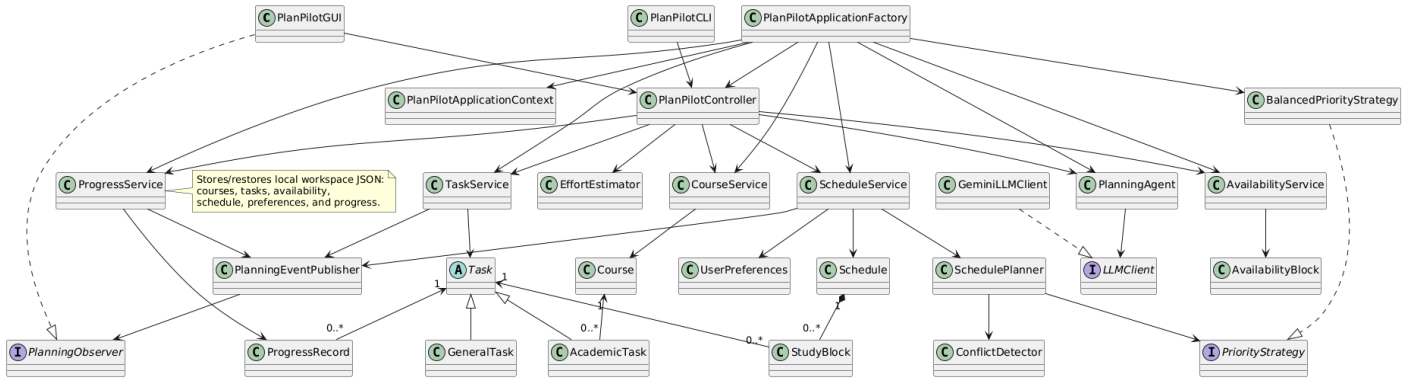
3.4 Adapter

Design problem: PlanPilot should not depend directly on a specific LLM provider or HTTP response shape.
Participants and roles: LLMClient is the provider-neutral interface; GeminiLLMClient adapts the Gemini REST API to generateText(); PlanningAgent depends on LLMClient.
Relevant methods: LLMClient.generateText(), GeminiLLMClient.generateText(), PlanningAgent.parseTask(), suggestEffort(), explainSchedule(), recovery/replan recommendation methods.
Why useful: AI-facing application code is isolated from provider-specific authentication, request/response JSON, HTTP errors, and model selection.
Stage 1 alignment: Modified. Stage 1 proposed OpenAIAdapter; Stage 2 implements GeminiLLMClient while preserving the same Adapter intent and LLMClient abstraction.

3.5 Observer

Design problem: Changes to tasks, schedules, preferences, or progress can affect what the GUI should display without tightly coupling services to JavaFX controls.
Participants and roles: PlanningObserver defines the observer callback; PlanPilotGUI is an observer; PlanningEventPublisher maintains subscribers; services publish planning-state changes.
Relevant methods: PlanningEventPublisher.subscribe(), unsubscribe(), notifyObservers(); PlanningObserver.onPlanningStateChanged().
Why useful: Services can announce state changes without knowing how the GUI renders them.
Stage 1 alignment: Matches Stage 1.

Updated UML Class Diagram (implemented architecture)



Affected sequence/design updates from Stage 1:

- F03: PlanningAgent now performs the Gemini request and TaskParser focuses on parsing/validation rather than directly owning the LLM dependency.
- F10: effort estimation is invoked directly from the Tasks workflow; a suggestion changes only the form until the user explicitly saves.
- F11: schedule explanations receive persisted generation evidence and priority-factor contributions from the actual deterministic plan.
- Application construction is centralized in PlanPilotApplicationFactory/ PlanPilotApplicationContext.

4. Changes and Rationale

OpenAI adapter -> GeminiLLMClient -> Gemini was selected as the available LLM provider -> The LLMClient Adapter boundary remained intact, so scheduling and domain code did not become provider-dependent.

TaskParser directly associated with LLM interaction -> PlanningAgent performs the LLM call and TaskParser parses/validates the structured response -> Clearer separation of concerns -> TaskParser is deterministic and easier to test.

Manual/technical GUI inputs -> Generated internal IDs, named selectors, conditional academic fields, and simplified workflows -> The original GUI exposed implementation details and was difficult to use -> The user-facing workflow became clearer without moving business logic into the GUI.

Separate AI effort panel -> F10 integrated into the Tasks form -> Effort estimation belongs to task creation/editing and Stage 1 described it in that context -> Users can request, review, keep, or apply a suggestion without navigating to a separate page or auto-saving a change.

Stage 1 recovery suggestions -> Explicit recovery actions with optional Gemini advice -> Recovery needed to be executable rather than explanatory only -> Users can add availability, extend the planning window, defer lower-priority work, or keep the partial plan, while every state-changing action still requires explicit user confirmation.
Basic schedule explanation context -> Persisted generation evidence and priority-factor contributions -> Gemini previously lacked enough evidence to explain some deterministic decisions reliably -> F11 explanations are more grounded while remaining read-only.

In-memory-only working state -> Local JSON workspace restoration through ProgressService/application construction -> Restarting the desktop application otherwise lost the user's planning state -> GUI and CLI can reopen the same local workspace. This is supporting infrastructure, not an additional counted feature.

Direct construction scattered across entry points -> PlanPilotApplicationFactory and PlanPilotApplicationContext -> GUI and CLI needed identical configured services and AI components -> Construction is centralized and both interfaces use the same architecture.

5. AI Tools and Collaboration Log

5.1 Implementation/development AI tools

OpenAI Codex
Model: GPT-6.1-Sol medium
Used for: implementation/refactoring assistance, JavaFX workflow fixes, Gemini integration, tests, persistence, schedule/progress bug fixes, and review of the current code.
Representative interactions included requests to implement Gemini through LLMClient, trace schedule-generation failures end-to-end, integrate adaptive effort estimation into the Tasks form, and fix GUI state/readiness issues without changing the domain architecture.
Token cost: 154000000
Money cost: included in subscription

5.2 Runtime LLM used by PlanPilot

Google Gemini API
Model: gemini-3.5-flash-lite

Used by the application for F03 natural-language task entry, F07 workload-recovery advice, F09 replan recommendations, F10 adaptive effort suggestions, and F11 schedule explanations.
Runtime cost/tokens: 150000
Money cost: Free
The real API key is stored locally and is not committed, included in screenshots, or written in this report.

6. Development and Verification Summary

6.1 Weekly commit summary

Period 1 — October 20-26, 2026
- 375c8ece — Implement Stage 1 domain model — https://github.com/cmell05/eecs3311/commit/375c8ece0181155e6460e24dd4c4252c4c2efcf4
- b26c73f7 — Implement course and task management services — https://github.com/cmell05/eecs3311/commit/b26c73f7e67985e0adc52542e7ad6b28002f8be9
- e8ec9aba — Implement availability and progress tracking — https://github.com/cmell05/eecs3311/commit/e8ec9abaa9b86f872ef32e879f259c2b98ca6b69
- f1a1f81a — Implement prioritization strategy and conflict detection — https://github.com/cmell05/eecs3311/commit/f1a1f81a2d7de5bb8cc373cc200c363085f13cc9
- 4d48aefb — Implement schedule generation and planning service — https://github.com/cmell05/eecs3311/commit/4d48aefb8f3f33fa377149d4df8eceefe5a0761c

Period 2 — October 27-November 2, 2026
- 63535ace — Implement PlanPilot controller facade — https://github.com/cmell05/eecs3311/commit/63535ace2993ddb258c60e3c31efcf96b51d8dcf
- 5381682e — Implement planning observer event system — https://github.com/cmell05/eecs3311/commit/5381682e7032d301c498559aebe7090f96aa1a67
- 2048143b — Implement PlanPilot command line interface — https://github.com/cmell05/eecs3311/commit/2048143b6ef07e2e19164bf493538008cedcfeb3
- 61f9a0be — Add JavaFX application shell and navigation — https://github.com/cmell05/eecs3311/commit/61f9a0be232bdf298054c8dd533083d49deb5002
- 973670d1 — Implement task, course, and availability GUI workflows — https://github.com/cmell05/eecs3311/commit/973670d111fbc47e7254cf06c4f28fb2cbb93e49

Period 3 — November 3-9, 2026
- 526952cb — Implement Gemini integration and adaptive effort estimation — https://github.com/cmell05/eecs3311/commit/526952cbbf0813c302cc4b149a567d8927d51417
- 170caf82 — Implement natural-language task parsing — https://github.com/cmell05/eecs3311/commit/170caf827910739886cb34b2123f684cca3f58ed
- 7805d2bb — Implement grounded AI schedule explanations — https://github.com/cmell05/eecs3311/commit/7805d2bb2f9f6df0d93e3acf47204743f0164237

- 4a8a6d24 — Implement AI workload recovery suggestions — https://github.com/cmell05/eecs3311/commit/4a8a6d243e5a34f895e0db97909fc5b6937a207a
- 3e8cd948 — Implement AI-assisted replanning recommendations — https://github.com/cmell05/eecs3311/commit/3e8cd948ad87e221046514de864aec40bf7887d8
All three reporting periods therefore contain at least five meaningful implementation/test commits.

6.2 Verification performed

- Maven unit/integration tests were run during implementation using mvn test / mvn clean test.
- The JavaFX GUI was exercised end-to-end for all F01-F11 feature demonstrations.
- The CLI was separately exercised to confirm that the command-line interface accesses the same controller/services and handles invalid input without terminating.
- Deterministic error cases demonstrated include duplicate/invalid data, invalid intervals, schedule infeasibility, progress corrections, and explicit replanning/recovery.
- Gemini-assisted actions were manually verified with a locally configured key; the key is excluded from Git through .env/.gitignore configuration.
- README launch/configuration instructions describe Java 17, Maven, GUI/CLI commands, and local Gemini configuration.

6.3 Important defects corrected

Important Stage 2 fixes included partial-schedule state handling, study-period preference handling, schedule invalidation/replanning behavior, generated IDs and course-reference protection, schedule/progress GUI wiring, AI Tool readiness state, task-form conditional fields, grounded schedule-explanation context, and persistence/restoration of planning state.

6.4 Remaining limitations

PlanPilot is local and single-user; it has no authentication or cloud synchronization. AI features depend on external Gemini availability. A transient Gemini failure may prevent an AI suggestion/explanation while deterministic planning continues to function. Stage 3 will contain the fuller deterministic/KUMA validation required by the overall project instructions.

7. Demonstration Index

Feature	Demonstration link	Notes
F01 — Course Management	https://youtu.be/jssAO9gPK7A	GUI feature demonstration
F02 — Task Management	https://youtu.be/C_jMAy0V6tg	GUI feature demonstration
F03 — Natural-Language Task Entry	https://youtu.be/NFJIo7skSKY	GUI feature demonstration
F04 — Availability and Commitment Management	https://youtu.be/TjN4LXFN8jw	GUIfeature demonstration
F05 — Task Prioritization	https://youtu.be/mc1_EMCXaBg	GUI feature demonstration
F06 — Study Schedule Generation	https://youtu.be/EfaGdMDhVTI	GUI feature demonstration
F07 — Schedule Conflict and Workload Recovery	https://youtu.be/F9MEcAeznek	GUI feature demonstration
F08 — Progress Tracking and Inferred Completion	https://youtu.be/xYJjJAr2yK0	GUI feature demonstration
F09 — Assisted Dynamic Replanning	https://youtu.be/huAK5cdWDDM	GUI feature demonstration
F10 — Adaptive Effort Estimation	https://youtu.be/D1yfUTm5tgk	GUI feature demonstration
F11 — Schedule Decision Explanation	https://youtu.be/O5jMsr6YMPo	GUI feature demonstration
CLI functionality	https://youtu.be/hNV8LSWZKdg	Separate CLI recording showing the command-line interface working

\